

# Training Quantised Neural Networks with STE Variants: the Additive Noise Annealing Algorithm

Matteo Spallanzani<sup>1\*</sup> Gian Paolo Leonardi<sup>2</sup> Luca Benini<sup>1,3</sup>

<sup>1</sup>Departement Informationstechnologie und Elektrotechnik, ETH Zürich, Switzerland

<sup>2</sup>Dipartimento di Matematica, [Università di Trento](#) Italy

<sup>3</sup>Dipartimento di Ingegneria Elettrica e dell'Informazione, [Università di Bologna](#) Italy

## Abstract

*Training quantised neural networks (QNNs) is a non-differentiable optimisation problem since weights and features are output by piecewise constant functions. The standard solution is to apply the straight-through estimator (STE), using different functions during the inference and gradient computation steps. Several STE variants have been proposed in the literature aiming to maximise the task accuracy of the trained network. In this paper, we analyse STE variants and study their impact on QNN training. We first observe that most such variants can be modelled as stochastic regularisations of stair functions; although this intuitive interpretation is not new, our rigorous discussion generalises to further variants. Then, we analyse QNNs mixing different regularisations, finding that some suitably synchronised smoothing of each layer map is required to guarantee pointwise compositional convergence to the target discontinuous function. Based on these theoretical insights, we propose additive noise annealing (ANA), a new algorithm to train QNNs encompassing standard STE and its variants as special cases. When testing ANA on the CIFAR-10 image classification benchmark, we find that the major impact on task accuracy is not due to the qualitative shape of the regularisations but to the proper synchronisation of the different STE variants used in a network, in accordance with the theoretical results.*

## 1. Introduction

Deep learning has rapidly advanced during the last decade over a wide range of domains, from computer vision to natural language processing [3, 8]. However, making it pervasive requires deploying deep neural networks (DNNs) on embedded or edge devices, i.e., on computing systems with limited storage, memory, and processing capabilities. These constraints are at odds with the typical requirements

of DNNs, which need millions or even billions of parameters and operations to deliver their performance.

Research in tiny machine learning (TinyML) has made considerable steps towards enabling the deployment of DNNs on resource-constrained devices. A first class of techniques aims at making DNNs more efficient in terms of accuracy-per-parameter or accuracy-per-operation [22, 23]. We refer to these techniques as *topological optimisations* since they revolve around achieving higher model efficiency by changing the structure of DNNs. A second class of approaches has instead focussed on deriving models that leverage the properties of the target deployment platform. These *hardware-related optimisations* include hardware-friendly activation functions [18], weight clustering and weight tensor decomposition [7, 13, 28], and QNNs [10].

QNNs use reduced-precision integer operands to meet the storage requirements and exploit the optimised support for integer arithmetic of embedded and edge platforms. With respect to their floating-point counterparts, QNNs typically introduce drops in task accuracy [11]. Several strategies have been proposed to counteract this shortcoming, ranging from changes to the target QNN's topology [16, 30], through mixing different data representations and precisions inside the same network [17, 24], to learning the shape of the stair functions used [4, 6, 12]. However, understanding how to propagate gradients through the discontinuous functions that model quantised operands remains a critical problem in QNN training.

**Background & related work** The classical derivative of a piecewise constant function is zero at all continuity points, while its distributional derivative is a linear combination of Dirac's deltas. Therefore, one can not directly apply the backpropagation algorithm to train a QNN. The standard solution to this problem is applying the so-called STE to all the discontinuous functions in the target QNN [2, 9]. Applying STE amounts to using two different functions during the forward and backward steps of the learning iteration,

\*Corresponding author: [spmatteo@ethz.ch](mailto:spmatteo@ethz.ch)